

تقرير التدقيق الهيكلي والأمني الشامل — Blind App (مُعِين)

Claude Code + Antigravity Orchestrator الفاحص:

النطاق: 45 ملف مصدري (~5300 سطر)

التاريخ: 21 أغسطس 2026

آخر Commit: c00d175

حاجب إطلاق رسمي (Launch Blocker): لا يجوز رفع التطبيق إلى متجر Google Play بحالته الراهنة؛ نظراً لاكتشاف تسريب مفاتيح توقيع مشفرة، وآلية ترخيص هشة سهلة الكسر، وانعدام تام لحقن الاعتماديات Dagger Hilt والاختبارات التلقائية.

📌 قرار وتوجيه الوكيل الرئيسي (Antigravity Orchestrator Decision):

اقترح فاحص Claude التخلي عن **Dagger Hilt** والاكتماء بالحقن اليدوي لأن حجم المشروع 45 ملفاً فقط. تم رفض هذا الاقتراح وتجاوزه تماماً التزاماً بقواعد المشروع الصارمة في **AGENTS.md** التي تنص على "الاعتماد الكلي على **Dagger Hilt** وفرض **Clean Architecture**". سنقوم بإدخال Hilt وعزل طبقة ال Domain تماماً في مرحلة إعادة الهيكلة.

🏠 البوابة 1 — المعمارية والكود النظيف

CRITICAL

C1. غياب كامل لـ Hilt والتوثيق يدعي وجوده

صفر استخدام لـ `@Inject` أو `@HiltAndroidApp`. يتم الاعتماد على Singletons يدوية تمنع استبدال الاعتماديات واختبار الكود.

CRITICAL

C2. انعدام طبقة ال Domain والـ UseCases

لا يوجد مجلد `domain/` ولا واجهة `QuranRepository` interface. المعمارية الفعلية MVVM بسيطة وليست Clean Architecture.

CRITICAL

C3. نموذج الـ God ViewModel الضخم (857 سطر)

`QuranViewModel.kt` يخلط 10 مسؤوليات (Media, Network) مع 7 StateFlows متداخلة (Haptics, TTS, Voice Commands, Bookmarks).

CRITICAL

C4. منطق I/O ثقيل متزامن في `{init}`

فتح `Keystore` و `EncryptedSharedPreferences` على الـ Main Thread في مُنشئ ViewModel يسبب خطر ANR ويمنع الـ Unit Tests.

HIGH

H2 + H3. كود ميت وتبعيات Retrofit و Firebase

طبقة الشبكة كاملة كود ميت غير مستدعي (المصدر محلي فقط)، وتبعيات Firebase BOM و Recaptcha مضمنة بلا أي استخدام فعلي.

🔥 الثغرات الأمنية الحرجة (حواجب الإطلاق)

CRITICAL

S1. تسريب كلمات مرور Keystore في Git

كلمات المرور مكتوبة صراحة كـ Fallback في `build.gradle.kts:44,50` ومحروقة في تاريخ Git. العلاج: تدوير المفتاح (Key Rotation) وحذف السر.

CRITICAL

S2. كشف رمز التفعيل وتجزئة PIN بدون ملح

التعليق يكشف رموز التفعيل نصاً في `TrialManager.kt:246`، وتجزئة SHA-256 Salt برقم 6 خانات تُكسر في أجزاء من الثانية عبر Rainbow Tables.

CRITICAL

S3. إيقاف التعطيم وتصغير الكود (R8 Minify)

`isMinifyEnabled = false` في بناء Release (`build.gradle.kts:63`)، ما يتيح هندسة التطبيق العكسية وقراءة شروط TrialManager بـ JADX بسهولة.

HIGH

S4. تسجيل طلبات الشبكة في الـ Release

`HttpLoggingInterceptor` يعمل على مستوى BODY بلا شرط `BuildConfig.DEBUG` في `NetworkModule.kt:13`.

HIGH

S5. تصدير MediaService بلا فحص onConnect

`QuranAudioService.kt` يتجاوز فقط `onMediaButtonEvent`، مما يسمح لأي تطبيق خارجي بالاتصال والتحكم بالصوت.

البوابة 2 — تغطية الاختبارات وجودتها

CRITICAL

T1. صفر اختبارات لـ QuranViewModel

الملف الأكثر تعقيداً (857 سطراً) يحتوي كل منطق التكرار والتنقل بلا أي اختبار وحدة نظراً لربطه المباشر بـ Android Context.

CRITICAL

T2. صفر اختبارات لـ TrialManager

آلة الحالة الزمنية الحساسة وكشف التلاعب بالساعة و SNTP لا تمتلك أي اختبار لحالات الحافة (Edge Cases).

CRITICAL

T3. اختبار وهمي (Tautological Test)

`ContinuousPlayLifecycleTest` يعيد كتابة الشرط المنطقي داخله بدلاً من استدعاء كود الإنتاج الحقيقي، مما يجعله أخضر دائماً حتى لو كُسر المنطق!

الطبقة / المكون	الحالة	ملاحظات الفحص
VoiceCommandParser	مغطى	25 تأكيداً مع MockK (المكون الوحيد المصمم بـ DI)
QuranViewModel	صفر	مستحيل الإنشاء بسبب ربطه بـ Keystore
TrialManager	صفر	منطق الأمان والترخيص بالكامل غير مختبر
QuranRepository	صفر	تطبيع النصوص العربية والبحث غير مختبر
Accessibility / UI	صفر	لا توجد اختبارات إمكانية وصول أو TalkBack

الأداء والتوثيق (البوابة 3)

PERFORMANCE

P1. حلقة تتبع تعيد رسم الشاشة 17 مرة/ثانية

`QuranViewModel:126` حلقة `delay(60L)` تحدث كائن `PlaybackUiState` الحاوي لـ 114 سورة بالكامل، مما يسبب `Recomposition` مستمر يستنزف البطارية.

PERFORMANCE

P2. قراءة وتحليل ملف المصحف كاملاً مع كل سورة

`QuranRepository.kt:98` يحلل ملف JSON الضخم عبر `JSONObject` لاستخراج سورة واحدة عند كل طلب ويكرر الكتابة في `Room`.

DOCS MISMATCH

D1 + D2. توثيق مضلل وتناقض مدة التجربة

التوثيق يدعي أن التجربة 7 أيام بينما الكود يعطي 30 يوماً (`... * 24 * 30L`), مع ادعاء استخدام `Hilt` غير الموجود.

DOCUMENTATION

D4. غياب KDoc في 78% من الملفات الحيوية

الملفات الأكثر تعقيداً مثل `QuranPlayerScreen` و `QuranAudioService` تحتوي على صفر توثيق `KDoc`.

خطة الإصلاح الهندسية المعتمدة (Implementation Roadmap)

المرحلة 1: المعمارية وحقن الاعتماديات (Hilt & Clean Architecture)

- تثبيت وتفعيل **Dagger Hilt** وحقن الاعتماديات بالكامل.
- إنشاء طبقة `domain` وتجريد `QuranRepository` interface.
- تفكيك `QuranViewModel` ونقل عمليات I/O الثقيلة إلى `Dispatchers.IO`.
- حذف التبعيات الميتة (`Retrofit`, `Firebase BOM`).

المرحلة 0: حواجب الإطلاق والأمان الفوري (Launch Blockers)

- تدوير مفاتيح التوقيع وإزالة كلمات المرور من `build.gradle.kts`.
- إصلاح ثغرة PIN وتشفيره بـ `Salt` و `PBKDF2/Argon2`.
- تفعيل تعقيم `R8 ProGuard` وحماية الـ `Release Build`.
- تقييد `onConnect` لخدمة الوسائط واستثناء الـ `Prefs` من الـ `Backup`.

المرحلة 3: هرم الاختبارات والتوثيق (Testing & KDoc)

- كتابة اختبارات وحدة شاملة لـ `TrialManager` و `QuranViewModel` عبر `MockK`.
- حذف وتصحيح الاختبارات الطوبولوجية الوهمية.
- موازمة التوثيق وحسم مدة التجربة في `PROJECT_CONTEXT.md`.

المرحلة 2: الأداء والبطارية (Performance Optimization)

- فصل `playbackProgress` إلى `Flow` مستقل وإيقافه في وضع الشاشة السوداء.
- تخزين المصحف في الذاكرة (`In-Memory Cache`) بعد الفرز الأول.
- تحرير `SimpleCache` بشكل سليم في `onDestroy`.